

Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ

ОБРАЗОВАТЕЛЬНОЕ

УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ

ИТМО

ITMO University

ЛАБОРАТОРНАЯ РАБОТА №4

По дисциплине Алгоритмы и структуры данных

Тема работы Алгоритмы поиска подстрок

Обучающийся Черненко Андрей Дмитриевич

Факультет Факультет прикладной информатики

Группа K4111c

Образовательная программа Разработка и эксплуатация промышленного
бэкенда

Обучающийся	_____	_____	<u>Черненко А. Д.</u>
	(дата)	(подпись)	(Ф.И.О.)

Руководитель	_____	_____	<u>Мусаев А. А.</u>
	(дата)	(подпись)	(Ф.И.О.)

Санкт–Петербург, 2026

Содержание

ВВЕДЕНИЕ	2
1 Задание 1. Наибольшая возрастающая последовательность	2
1.1 Условие	2
1.2 Алгоритм решения	2
1.3 Результат работы	3
2 Задание 2. Поиск двузначных чисел в строке	4
2.1 Условие	4
2.2 Формирование строки	4
2.3 Наивный алгоритм	5
2.4 Алгоритм Рабина–Карпа	5
2.5 Алгоритм Бойера–Мура	6
2.6 Алгоритм Кнута–Морриса–Пратта	6
2.7 Сравнение алгоритмов	7
ЗАКЛЮЧЕНИЕ	7

ВВЕДЕНИЕ

Целью лабораторной работы является изучение алгоритмов поиска подстрок и их практическое применение. В первой части требуется найти наибольшую непрерывную возрастающую последовательность в массиве целых чисел. Во второй части необходимо сформировать строку из 500 чисел и определить наиболее часто встречающиеся двузначные числа с помощью нескольких алгоритмов поиска.

Так как номер варианта нечётный, в работе используются числа Фибоначчи. Для поиска подстрок реализованы наивный алгоритм, алгоритм Рабина–Карпа, алгоритм Бойера–Мура и алгоритм Кнута–Морриса–Пратта.

1 Задание 1. Наибольшая возрастающая последовательность

1.1 Условие

Дан массив из n случайных целых чисел в диапазоне от -100 до 100 . Необходимо найти наибольшую непрерывную возрастающую последовательность, то есть серию, для которой выполняется:

$$N[i] < N[i + 1] < \dots < N[i + m].$$

1.2 Алгоритм решения

Массив просматривается слева направо. Переменная `currentLength` хранит длину текущей возрастающей серии. Если очередной элемент больше предыдущего, длина серии увеличивается. Иначе начинается новая серия длины один. При каждом увеличении проверяется, стала ли текущая серия длиннее найденной ранее.

```
1 private static (int Start, int Length)
2     FindLongestIncreasingSequence(int[] numbers)
3 {
4     if (numbers.Length == 0) return (-1, 0);
5
6     var bestStart = 0;
7     var bestLength = 1;
8     var currentStart = 0;
9     var currentLength = 1;
```

```

10
11     for (var i = 1; i < numbers.Length; i++)
12     {
13         if (numbers[i - 1] < numbers[i])
14             currentLength++;
15         else
16         {
17             currentStart = i;
18             currentLength = 1;
19         }
20
21         if (currentLength > bestLength)
22         {
23             bestStart = currentStart;
24             bestLength = currentLength;
25         }
26     }
27
28     return (bestStart, bestLength);
29 }

```

Листинг 1: Поиск наибольшей возрастающей серии

Последовательный просмотр массива: сравнение $N[i - 1]$ и $N[i]$ — продолжение текущей серии или начало новой.

Рисунок 1 - Схема поиска возрастающей серии

Алгоритм выполняет один проход по массиву, поэтому его временная сложность равна $O(n)$, а дополнительная память — $O(1)$.

1.3 Результат работы

Для воспроизводимости используется генератор случайных чисел с фиксированным начальным значением. Программа формирует массив из 20 чисел в диапазоне $[-100; 100]$, выводит его, длину найденной серии, сами элементы и их индексы.

В результате запуска был получен массив

$\{-68, -44, 84, 99, 84, -85, 34, 71, 26, -27, 82, -61, -18, 1, -75, -38, 2, -78, 4, 59\}$.

Наибольшая возрастающая серия имеет длину 4: $\{-68, -44, 84, 99\}$.

2 Задание 2. Поиск двузначных чисел в строке

2.1 Условие

Необходимо заполнить строку 500 числами Фибоначчи, записанными слитно. Затем для каждого двузначного числа от 10 до 99 требуется посчитать количество вхождений в строку, используя наивный алгоритм, Рабина–Карпа, Бойера–Мура и Кнута–Морриса–Пратта. Вхождения, начинающиеся на соседних позициях, считаются отдельно.

2.2 Формирование строки

Числа Фибоначчи строятся рекуррентно:

$$F_0 = 0, \quad F_1 = 1, \quad F_i = F_{i-1} + F_{i-2}.$$

Для хранения больших значений используется тип `BigInteger`, после чего числа последовательно добавляются в объект `StringBuilder`.

```
1 private static string BuildFibonacciText()
2 {
3     var text = new StringBuilder();
4     BigInteger previous = 0;
5     BigInteger current = 1;
6
7     for (var i = 0; i < 500; i++)
8     {
9         text.Append((i == 0 ? previous : current).ToString());
10        (previous, current) = (current, previous + current);
11    }
12
13    return text.ToString();
14 }
```

Листинг 2: Формирование строки из чисел Фибоначчи

0, 1, 1, 2, 3, 5, 8, 13, 21, ... → последовательное объединение строк → строка для поиска подстрок.

Рисунок 2 - Формирование строки из чисел Фибоначчи

2.3 Наивный алгоритм

Наивный алгоритм проверяет каждую возможную позицию начала подстроки и посимвольно сравнивает образец с соответствующим фрагментом текста.

```
1 private static int NaiveCount(string text, string pattern)
2 {
3     var count = 0;
4     for (var start = 0;
5         start <= text.Length - pattern.Length; start++)
6     {
7         var matched = true;
8         for (var offset = 0; offset < pattern.Length; offset++)
9         {
10             if (text[start + offset] == pattern[offset])
11                 continue;
12             matched = false;
13             break;
14         }
15
16         if (matched) count++;
17     }
18
19     return count;
20 }
```

Листинг 3: Наивный поиск всех вхождений

Временная сложность наивного алгоритма равна $O(nm)$, где n — длина текста, а m — длина образца. Дополнительная память равна $O(1)$.

2.4 Алгоритм Рабина–Карпа

Рабин–Карп сравнивает хеш образца с хешем каждого окна текста. При совпадении хешей выполняется дополнительная проверка символов, что защищает от коллизий.

В работе используется полиномиальный скользящий хеш:

$$H(s) = \sum_{i=0}^{m-1} s_i B^{m-1-i} \pmod{q}.$$

В среднем алгоритм работает за $O(n + m)$, а в худшем случае из-за коллизий может деградировать до $O(nm)$.

2.5 Алгоритм Бойера–Мура

Бойер–Мур сравнивает образец справа налево и использует таблицу сдвигов по последнему символу окна. Если найдено несовпадение, окно сдвигается сразу на несколько позиций.

Для небольшого алфавита и коротких образцов практическая эффективность алгоритма высока. Средняя сложность обычно близка к $O(n/m)$, а худшая сложность базовой реализации составляет $O(nm)$. Память на таблицу сдвигов постоянна относительно текста.

2.6 Алгоритм Кнута–Морриса–Пратта

КМП предварительно строит таблицу префикс-функции образца. При несовпадении уже проверенная часть не просматривается заново: алгоритм использует длину совпавшего префикса.

```
1 private static int KnuthMorrisPrattCount(  
2     string text, string pattern)  
3 {  
4     var prefix = BuildPrefixTable(pattern);  
5     var count = 0;  
6     var matched = 0;  
7  
8     for (var i = 0; i < text.Length; i++)  
9     {  
10         while (matched > 0 && text[i] != pattern[matched])  
11             matched = prefix[matched - 1];  
12         if (text[i] == pattern[matched]) matched++;  
13  
14         if (matched != pattern.Length) continue;  
15         count++;  
16         matched = prefix[matched - 1];  
17     }  
18  
19     return count;  
20 }
```

Листинг 4: Подсчёт вхождений алгоритмом КМП

Построение таблицы и поиск выполняются за $O(m)$ и $O(n)$ соответственно. Итоговая сложность равна $O(n + m)$, а дополнительная память — $O(m)$.

2.7 Сравнение алгоритмов

Таблица 1 - Сравнение алгоритмов поиска подстрок

Алгоритм	Средняя сложность	Худшая сложность	Особенности
Наивный	$O(nm)$	$O(nm)$	Простая реализация, не требует предварительной обработки
Рабин–Карп	$O(n + m)$	$O(nm)$	Использует хеширование, возможны коллизии
Бойер–Мур	$O(n/m)$ на практике	$O(nm)$	Большие сдвиги при несовпадении справа
КМП	$O(n + m)$	$O(n + m)$	Гарантированное время, нужна таблица префиксов

Программа запускает каждый алгоритм для всех 90 двузначных образцов и проверяет, что результаты совпадают. Дополнительно выводится время выполнения в тактах таймера. Наиболее часто встречающееся двузначное число определяется как образец с максимальным количеством найденных вхождений.

В сформированной строке длиной 26252 символа наиболее часто встречающимся двузначным числом оказалось 71, которое встретилось 297 раз. Все четыре алгоритма дали одинаковый результат.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы была реализована линейная процедура поиска наибольшей непрерывной возрастающей последовательности. Алгоритм выполняет единственный проход по массиву и использует постоянный объём дополнительной памяти.

Для второй части была сформирована строка из 500 чисел Фибоначчи. Подсчёт двузначных подстрок выполнен четырьмя алгоритмами: наивным, Рабина–Карпа, Бойера–Мура и Кнута–Морриса–Пратта. Все алгорит-

мы должны выдавать одинаковые частоты, однако отличаются количеством предварительной обработки, используемой памятью и гарантией времени работы.

Наивный алгоритм проще всего реализовать и удобно использовать для коротких образцов. Рабин–Карп полезен при массовом поиске благодаря хешированию, но чувствителен к коллизиям. Бойер–Мур часто показывает лучшую практическую скорость за счёт больших сдвигов. КМП обеспечивает гарантированную линейную сложность и подходит для случаев, когда важна предсказуемость времени выполнения.